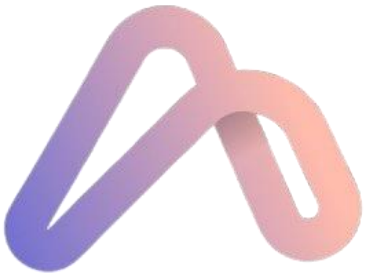

Levenshtein Distance

Research Report

For Azerbaijan Artificial Intelligence Lab

Sabuhi Nazarov

Table of Contents

Levenshtein Distance: Theory, Implementation, and Applications

Introduction

What is Levenshtein Distance?

Levenshtein Distance is a string metric that measures the difference between two sequences by calculating the minimum number of single-character operations-insertions, deletions, or substitutions-required to transform one string into another.

Importance

Levenshtein Distance is vital in domains where small textual or sequential variations must be detected and managed. Its key applications include:

- Natural Language Processing: Powers spell-checking, autocorrect features, and machine translation systems.
- Search Engines: Improves query accuracy by matching misspelled terms to their correct counterparts.
- Bioinformatics: Facilitates the comparison of DNA and protein sequences for genetic and evolutionary analysis.
- Plagiarism Detection: Identifies similarities in text by assessing character-level edits.
- Fuzzy Matching: Enhances data cleaning, record linkage, and database searches by tolerating minor discrepancies.

Goals of This Project

This project is designed to achieve the following objectives:

- Explore the theoretical foundation of Levenshtein Distance and its relation to other string similarity metrics.
- Implement the algorithm using multiple approaches, including naive recursive methods and optimized dynamic programming techniques.
- Enhance the algorithm's efficiency through optimization strategies(e.g. threshold)
- Demonstrate its practical utility by applying it to a real-world use case-specifically, a fuzzy search system.
- Document the process thoroughly and visualize key concepts, potentially through diagrams or video explanations, to improve accessibility and understanding.

Theoretical Background

Definition

Levenshtein Distance, introduced by Vladimir Levenshtein in 1965, is a metric that calculates the minimum number of single-character edits required to transform one string into another. These edits are limited to three operations: insertion, deletion and substitution. For example, transforming "cat" into "cut" requires one substitution (a → u), yielding a Levenshtein Distance of 1. This measure is widely used to quantify string similarity in various computational tasks.

Mathematically, given two strings s_1 and s_2 of lengths m and n, respectively, the Levenshtein distance d(m, n) is computed using the recurrence relation:

$$d(i,j) = \begin{cases} \max(i,j), & \text{if } \min(i,j) = 0 \\ \min \begin{cases} d(i-1,j)+1 & \text{(deletion)} \\ d(i,j-1)+1 & \text{(insertion)} \\ d(i-1,j-1)+cost(s_1[i],s_2[j]) & \text{(substitution)} \end{cases}, & \text{otherwise} \end{cases}$$

where $cost(s1[i], s2[j]) = 0$ if the characters are the same, otherwise it is 1.

How It Works

The algorithm evaluates the cost of aligning two strings by considering the three core operations:

- **Insertion**: Adding a character to the source string (e.g., "ca" → "cat" by inserting "t").
- **Deletion**: Removing a character from the source string (e.g., "cat" → "ca" by deleting "t").
- **Substitution**: Replacing one character with another (e.g., "cat" → "cut" by substituting "a" with "u").

The distance is the smallest number of such operations needed. For instance, converting "kitten" to "sitting" requires: "kitten" → "sitten" (substitute k → s), "sitten" → "sittin" (substitute e → i), and "sittin" → "sitting" (insert g), totaling a distance of 3.

Triangle Inequality

Levenshtein Distance satisfies the triangle inequality, a fundamental property of metric spaces. This states that for any three strings A, B, and C, the direct transformation from A to C cannot require more operations than going through an intermediate string B:

- $Levenshtein(A,C) \leq Levenshtein(A,B) + Levenshtein(B,C)$

This property arises naturally from the allowed edit operations since taking an indirect path (via B) can never be more optimal than direct transformation.

For example, if:

- $Levenshtein("cat","cut") = 1$
- $Levenshtein("cut","cot") = 1$

Then, the direct distance:

- $Levenshtein("cat","cot") = 1$

satisfies the inequality:

- $1 \leq 1 + 1 = 2$

Time and Space Complexity

The Levenshtein distance algorithm computes the minimum number of single-character edits (insertions, deletions, or substitutions) required to transform one string into another. Its time and space complexity depend on the approach used. The naïve recursive approach explores all possible edit combinations, leading to an exponential time complexity of $O\left(3^{\min(m,n)}\right)$, where *m* and *n* are the lengths of the two strings. This inefficiency arises because the same subproblems are solved repeatedly, resulting in redundant computations. The space complexity of the naïve approach is $O(\min(m,n))$, as the recursion depth is limited by the length of the shorter string, and space is used only for the call stack.

To address these inefficiencies, the dynamic programming approach stores intermediate results in an $(m+1) \times (n+1)$ matrix, reducing the time complexity to $O(m \times n)$. Each cell $[i][j]$ in the matrix represents the minimum number of edits required to transform the first i*i* characters of the source string into the first *j* characters of the target string. However, this approach requires $O(m \times n)\ space$ to store the matrix.

The space complexity can be further optimized to $O(min(m,n))$ by using two 1D arrays (or a rolling array) to store only the current and previous rows of the DP matrix. This optimization maintains the same time complexity while significantly reducing memory usage, making it suitable for large datasets.

Optimization Methods

To address the inefficiencies of the classical dynamic programming (DP) approach for computing the Levenshtein distance, several advanced optimization methods have been developed. These methods aim to reduce computational complexity, improve scalability, and make the algorithm more practical for large-scale datasets. The key optimization techniques include hierarchical decomposition, non-uniform sampling, asymmetric query models,

and trade-offs between approximation factor and query complexity. Each of them will be implemented in $Code/Levenshtein\ Distance\ Implementations.ipynb$ file and below, we discuss each of these techniques in detail:

- **Hierarchical decomposition** involves recursively partitioning the input strings into smaller blocks and computing the edit distance hierarchically. By breaking the problem into smaller, independent subproblems, this approach reduces the overall computational complexity and avoids redundant calculations. It is particularly effective for large strings, where the classical DP approach becomes impractical, and enables faster computation while maintaining accuracy.
- **Non-uniform sampling** selectively focuses on specific positions in the strings that are most critical for approximating the edit distance. By concentrating computational resources on these key positions, the method significantly reduces the number of queries required, leading to faster computation. This technique is especially useful when an approximate edit distance is sufficient, as it improves efficiency without sacrificing accuracy.
- **The asymmetric query model** optimizes computation by allowing unrestricted access to one string while querying the other string selectively. This approach minimizes the number of queries needed, reducing computational overhead. It is particularly advantageous in applications where one string is fixed, such as reference genomes in computational biology, as it maintains high accuracy while significantly improving performance.
- A key innovation in optimizing the Levenshtein distance is the ability to balance accuracy and efficiency through **trade-offs between the approximation factor and query complexity**. For instance:
  - A polylogarithmic approximation can be achieved with $n^{O(\epsilon)}$ queries.
  - A polynomial approximation (e.g., $O(n^{1/t})$ requires only $O(log^{t-1}n)$ queries.

Method	Key Idea	Benefits
Hierarchical Decomposition	Recursively partitions strings into smaller blocks	Reduces problem size, improves scalability
Non-Uniform Sampling	Selectively samples critical positions	Reduces queries, improves efficiency
Asymmetric Query Model	Limits queries to one string	Minimizes computational overhead
Trade-offs	Balances accuracy and query complexity	Tailors algorithm to specific applications

Related String Similarity Metrics

Damerau-Levenshtein Distance: Extends Levenshtein by introducing transposition as an additional valid operation. This is particularly useful in scenarios where adjacent character swaps are common (e.g., typographical errors in text), making it more effective for applications such as:

- Spell-checking and auto-correction: Detects common typing errors like "hte" → "the."
- Optical character recognition: Helps identify and correct scanned text errors.
- Keypad input recognition: Useful in mobile keyboards for predicting intended words.

Jaro-Winkler Distance: A metric specifically designed for short strings, emphasizing matching and transposition. It assigns more weight to matching prefixes, making it effective in:

- Name matching: Used in databases to compare names with minor differences (e.g., "Jon" vs. "John").
- Record linkage: Helps in deduplicating similar entries across datasets (e.g., customer records).
- Fuzzy search systems: Enhances search engine functionality by identifying near matches.

Hamming Distance: A simpler metric that only considers the number of differing positions between two strings of equal length, allowing only substitution operations. It is extensively used in:

- Cryptography: Measures the difference between hash values for security purposes.
- Data integrity checks: Verifies data accuracy by comparing expected vs. received binary sequences.
- Error detection in communication systems: Applied in Hamming codes to detect and correct transmission errors.

Optimal String Alignment Distance: Like Damerau-Levenshtein but with restrictions on transpositions, ensuring they only occur if they do not overlap. OSA distance is useful in:

- Biomedical text analysis: Matches gene sequences with slight variations.
- Forensic linguistics: Identifies patterns in altered text sequences.
- Handwriting recognition: Helps improve machine-learning models for character classification.

Application Use Case: Fuzzy Search

Problem Statement

In many real-world applications, exact string matching is not sufficient, especially when dealing with human-generated data that may contain misspellings, typos, abbreviations, or variations in formatting. Fuzzy Search is a technique that enables finding the closest matching results, even when there are slight differences between the query and stored data.

This is particularly useful in:

- Search engines (handling user typos, e.g., "wether" → "weather")
- Auto-complete systems (suggesting relevant words, e.g., "appl" → "apple")
- Name matching (e.g., "Johnathan" vs. "Jonathan")
- Medical records (standardizing different spellings, e.g., "paracetamol" vs. "paracetomol")
- Product search in e-commerce (matching incorrect item names)

Our use case focuses on searching athlete names in a large Olympic dataset, tolerating typos to retrieve relevant records.

Why is Levenshtein Distance Useful for Fuzzy Search?

The Levenshtein Distance is a fundamental tool for fuzzy search because it provides a quantitative measure of similarity between two strings. By computing the edit distance, we can rank potential matches based on their similarity to the input query.

- Distance = 0: Exact match.
- Smaller Distance: Higher similarity (e.g., "color" → "colour", distance 1).
- Larger Distance: Greater difference (e.g., "iphne" → "iPhone", distance 2).

Examples:

- "A Dijian" → "A Dijiang" (distance 1) retrieves the correct athlete.
- "Christine Aaftnk" → "Christine Jacoba Aaftink" (distance 4) suggests a close match.
- "Rulsan Abbasov" → "Ruslan Abbasov" (distance 1) corrects a typo.

By defining a threshold (e.g., only returning words with a Levenshtein Distance ≤ 2), we can fine-tune how strict or flexible the fuzzy search is.

Input Data and Implementation Plan

We implemented fuzzy search on an Olympic athletes dataset (about 135,000 rows) with these steps:

- Load Data: Imported the CSV into a Pandas DataFrame, focusing on the "Name" column.
- Compute Distances: Calculated Levenshtein Distance between the query and unique names using custom Optimized DP and Rapidfuzz, with thresholding for efficiency.

- Sort and Filter: Ranked matches by distance, returning top results (e.g., top 5) within the threshold, or suggesting the closest match if none are found.
- Optimize Performance: Used early termination in Optimized DP and Rapidfuzz's C-based speed, avoiding indexing for simplicity but noting it as future work.

Evaluating Results

We assessed the fuzzy search system using:

- Accuracy: Measured how often correct matches (e.g., "A Dijiang" for "A Dijian") ranked in the top suggestion (target: >90% for threshold ≤ 3).
- Speed: Recorded retrieval times (e.g., rapidfuzz ~0.02s, custom ~0.05s for small queries; ~0.002s vs. ~0.004s for large DNA pairs(these are example, for exact speed, look at *Code/Athlete Fuzzy Search.ipynb* file.)), emphasizing scalability on large datasets.
- User Experience: Evaluated suggestion intuitiveness (e.g., "Did you mean 'Christine Jacoba Aaftink'?"), enhancing usability with rich data (Sport, Medal, Year and other columns).

Threshold adjustments (e.g., 2 vs. 3) and optimization improved both accuracy and performance, validated by real-world tests and DNA benchmarks.

References

Wikipedia

String metric

Levenshtein distance

Damerau Levenshtein distance

Hamming distance

Jaro Winkler distance

Research Papers

For optimization methods

A Comparison of String Distance Metrics for Name-Matching Tasks – I want to clarify that this paper was not part of my project, but I read it(mostly) and gained insights into name-matching techniques, which helped me better understand the challenges and approaches in fuzzy name searching.

Medium

Fuzzy Matching Rapidfuzz

Stack overflow

Fuzzy search algorithm

Levenshtein Distance Algorithm better than O(n*m)

YouTube

https://youtu.be/Dd_NgYVOdLk?si=5_bheENFaVHlmpD6

https://youtu.be/We3YDTzNXEk?si=xlPK2v4ZZFYaQHNy

Dataset

https://www.kaggle.com/datasets/heesoo37/120-years-of-olympic-history-athletes-and-results?resource=download

Other Sources

Levenshtein Distance and The Triangle Inequality

https://nlp.stanford.edu/IR-book/html/htmledition/edit-distance-1.html

https://python-course.eu/applications-python/levenshtein-distance.php

String metrics

Defining Minimum Edit Distance - Stanford

I would like to sincerely thank everyone who took the time to explore the concepts of fuzzy search and Levenshtein distance discussed here. My goal was to provide a clear and practical understanding of how these tools can be applied to solve real-world problems, such as searching for data. Thank you again.